
# Fuel Fast Bridge

Security Assessment

November 28th, 2025 — Prepared by OtterSec

Alpha Toure shxdow@osec.io

James Wang james.wang@osec.io

# Table of Contents


**Executive Summary** **3**

Overview 3

Key Findings 3

**Scope** **4**

**Findings** **5**

**Vulnerabilities** **6**

OS-FFB-ADV-00 | Premature Withdrawal Recording 8

OS-FFB-ADV-01 | Inability to Fetch Partially Processed Block 9

OS-FFB-ADV-02 | Edge Case on Fetching New Withdrawals 10

OS-FFB-ADV-03 | Floating-Point Precision Calculation Risk 11

OS-FFB-ADV-04 | Failure to Decrement Relayer Count 12

OS-FFB-ADV-05 | Inconsistent Withdrawal Hash Encoding 13

OS-FFB-ADV-06 | Incomplete Withdrawal Event Verification 14

OS-FFB-ADV-07 | Race Condition on UTXO Spending due to Shared Signer 15

OS-FFB-ADV-08 | Possibility of Stale oracle configuration 16

OS-FFB-ADV-09 | Hardcoded Fuel Chain ID 17

OS-FFB-ADV-10 | Incorrect Decimal Precision Handling 18

**General Findings** **19**

OS-FFB-SUG-00 | Indexer Event Reliability Concerns 20

OS-FFB-SUG-01 | Ineffective Transaction Limit Enforcement 21

OS-FFB-SUG-02 | Automatic Threshold Adjustment Risk 22

OS-FFB-SUG-03 | Code Maturity 23

OS-FFB-SUG-04 | Code Refactoring 25

OS-FFB-SUG-05 | Code Inconsistencies 27

**Vulnerability Rating Scale 28**

**Procedure 29**

# 01 — Executive Summary

## Overview

Fuel Labs engaged OtterSec to assess the `fuel-fast-bridge` program. This assessment was conducted between August 15th and September 4th, 2025. For more information on our auditing methodology, refer to chapter 07.

## Key Findings

We produced 17 findings throughout this audit engagement.

In particular, we we discovered inconsistencies in the withdrawal logic including incorrectly marking withdrawals as processed before they are approved or executed (OS-FFB-ADV-00), and the possibility of a relayer getting stuck if multiple withdrawals occur in a single block exceeding the batch size, as the query only advances by the last processed block height (OS-FFB-ADV-02). Also, the withdrawal event verification function only verifies the first withdrawal event in a transaction, potentially missing other events, which may allow unverified withdrawals to pass undetected (OS-FFB-ADV-06).

Additionally, the decimals variable incorrectly returns the remote token's decimals instead of Fuel's fixed 9-decimal format (OS-FFB-ADV-10), and the deposit and gas oracle updates share the same signers, which may create race conditions when fetching UTXOs for gas spending, resulting in transaction failures (OS-FFB-ADV-07).

Furthermore, the fuel chain ID is hardcoded to zero, resulting in deposits to always reference the testnet (OS-FFB-ADV-09), and the offchain gas publisher may utilize outdated parameters as it does not re-fetch updated oracle configuration (OS-FFB-ADV-08).

We also recommended refactoring the codebase for improved functionality and efficiency (OS-FFB-SUG-04), and made suggestions regarding ensuring adherence to coding best practices (OS-FFB-SUG-03). We further highlighted several discrepancies and ambiguities in the exiting codebase that should be properly addressed for consistency and clarity (OS-FFB-SUG-05).

# 02 — Scope

The source code was delivered to us in a git repository at https://github.com/FuelLabs/fuel-fast-bridge. This audit was performed against commit 2c06263.

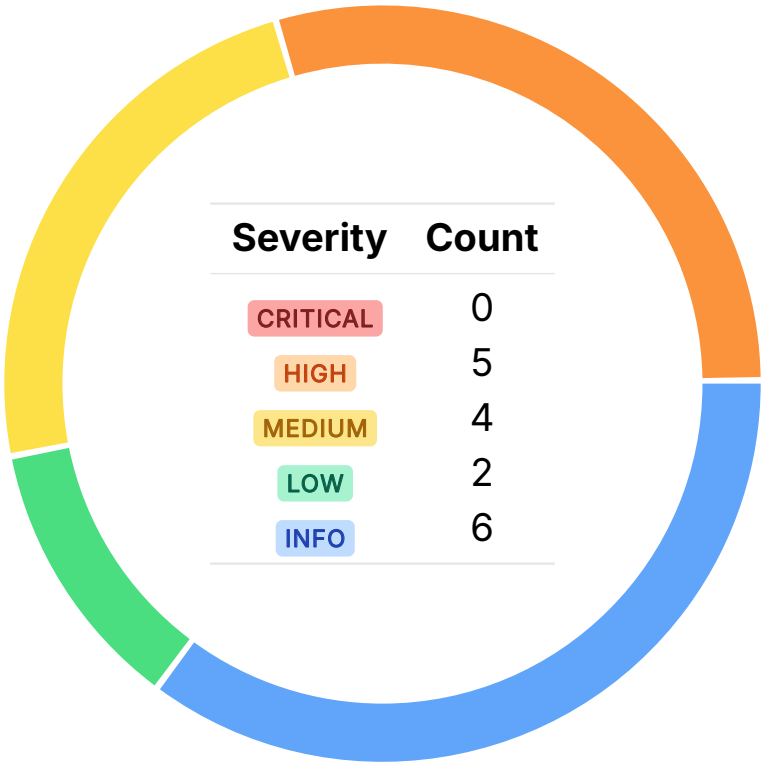
**A brief description of the program is as follows:**

| Name | Description |
| --- | --- |
| fuel-fast-bridge | A cross-chain bridge facilitating secure asset transfers between EVM-compatible chains (Base/Ethereum) and the Fuel blockchain. It uutilizes a relayer-based architecture with custom multi-sig contracts, configurable transaction limits, universal wrapped assets, and monitoring capabilities. |

# 03 — Findings

Overall, we reported 17 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



| Severity | Count |
| --- | --- |
| CRITICAL | 0 |
| HIGH | 5 |
| MEDIUM | 4 |
| LOW | 2 |
| INFO | 6 |

# 04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

| ID | Severity | Status | Description |
|---|---|---|---|
| OS-FFB-ADV-00 | HIGH | RESOLVED | Withdrawals are incorrectly marked as processed in `initializeFromLatestWithdrawals` before they are approved or executed. |
| OS-FFB-ADV-01 | HIGH | RESOLVED | `pollSentioAPI` utilizes `>` for block height filtering when querying the Sentio API for new withdrawal events, which may skip partially processed blocks. |
| OS-FFB-ADV-02 | HIGH | RESOLVED | The relayer may get stuck if multiple withdrawals occur in a single block exceeding the batch size, as the query only advances by `lastProcessedBlockHeight` in `fetchNewWithdrawals`. |
| OS-FFB-ADV-03 | HIGH | RESOLVED | The current code utilizes TypeScript's floating-point `number` type, which may create precision errors in calculations, resulting in incorrect amounts or comparison results. |
| OS-FFB-ADV-04 | HIGH | RESOLVED | `remove_relayer` removes a relayer's address but does not update `relayer_count`, resulting in an inconsistent state and potential quorum miscalculations. |

| | | | |
|---|---|---|---|
| OS-FFB-ADV-05 | MEDIUM | RESOLVED | `generateRequestHash` utilizes `AbiCoder.encode` instead of `encodePacked`, resulting in mismatched hash outputs between indexers and on-chain events. |
| OS-FFB-ADV-06 | MEDIUM | RESOLVED | `verifyWithdrawalEvent` verifies only the first withdrawal event in a transaction, potentially missing other events, which may allow unverified withdrawals to pass undetected. |
| OS-FFB-ADV-07 | MEDIUM | RESOLVED | The deposit and gas oracle updates share the same signers, which may create race conditions when fetching UTXOs for gas spending, resulting in transaction failures. |
| OS-FFB-ADV-08 | MEDIUM | RESOLVED | The offchain gas publisher may utilize outdated parameters, as it does not refetch the updated oracle configuration. |
| OS-FFB-ADV-09 | LOW | RESOLVED | The `fuel_chain_id` is hardcoded to zero, resulting in deposits to always reference the testnet. |
| OS-FFB-ADV-10 | LOW | RESOLVED | `decimals` incorrectly returns the remote token's decimals instead of Fuel's fixed 9-decimal format. |

## Premature Withdrawal Recording HIGH OS-FFB-ADV-00

### Description

`initializeFromLatestWithdrawals` in `withdrawals.task` incorrectly records withdrawals in the `processed_withdrawal` table before they are approved by the relayer or confirmed as executed on-chain. As a result of this premature recording, the system treats these pending withdrawals as completed, preventing their proper processing.

>_ *src/tasks/withdrawals.task.ts* TYPESCRIPT

```typescript
async function initializeFromLatestWithdrawals([...]): Promise<WithdrawalState | null> {
  logger.info(PROCESS, 'Finding resumption point from latest withdrawals');
  try {
    [...]
    // Batch record all unprocessed withdrawals
    if (unprocessedWithdrawals.length > 0) {
      logger.info(PROCESS, `Recording ${unprocessedWithdrawals.length} unprocessed
          ↪  withdrawals`);

      const processedWithdrawals = unprocessedWithdrawals.map((withdrawal) => ({[...]}));

      const success = dbService.batchRecordProcessedWithdrawals(processedWithdrawals);
      if (!success) {
        logger.error(PROCESS, 'Failed to batch record withdrawals');
        return null;
      }
    }
    [...]
  } catch (error) {
    logger.error(PROCESS, 'Initialization error:', error);
    return null;
  }
}
```

### Remediation

Ensure that withdrawals are only recorded in `processed_withdrawal` after they have been approved by the relayer or successfully executed.

### Patch

Resolved in PR#263.

## Inability to Fetch Partially Processed Block HIGH OS-FFB-ADV-01

### Description

`pollSentioAPI` in `withdrawals.task` utilizes a strict `>` condition when it queries the Sentio API for new withdrawal events. This implies the relayer will only fetch events from blocks strictly greater than the last processed block number. It assumes that all events in the previous block were processed completely and successfully. Unfortunately, this assumption is not upheld by envio, and thus the relayer might miss withdrawals if blocks contain too many withdrawal events.

Thus, if an interruption occurs midway through processing events within a block, the `lastProcessedBlockNumber` will still be updated to that block number (even if not all events were handled). On the next polling cycle, because the SQL query utilizes `>`, the relayer skips re-fetching events from that partially processed block.

*>_ src/tasks/withdrawals.task.ts* TYPESCRIPT

```typescript
async function pollSentioAPI(
  apiKey: string,
  apiUrl: string,
  chainManager: ChainManager,
  environment: string,
  verificationProvider: Provider | null
) {
  try {
    const tableName = environment === 'e2e-testnet' ? 'FastBridgeWithdrawal_E2E' :
        ↪  'FastBridgeWithdrawal';
    const sqlQuery =
      withdrawalState.lastProcessedBlockNumber === 0
        ? `SELECT * FROM \`${tableName}\` ORDER BY block_number ASC`
        : `SELECT * FROM \`${tableName}\` WHERE block_number >
            ↪  ${withdrawalState.lastProcessedBlockNumber} ORDER BY block_number ASC`;
    [...]
  } catch (error) {
    logger.error(PROCESS, `Error polling Sentio API:`, error);
    throw error;
  }
}
```

### Remediation

Change the condition to `>=` to ensure re-fetching of the last processed block.

### Patch

Resolved in PR#272.

## Edge Case on Fetching New Withdrawals HIGH OS-FFB-ADV-02

### Description

`fetchNewWithdrawals` in `EnvioService` fetches the first `batchsize` of events starting from `lastProcessedBlockHeight`, which fails when multiple withdrawal events exist within the same block. If their count exceeds the configured `batchSize`, only the first batch is ever fetched, rendering the remaining events unprocessed, as `lastProcessedBlockHeight` will never advance. This stalls the relayer indefinitely on that block height, halting further withdrawal processing.

>_ *src/infra/services/EnvioService.ts* TYPESCRIPT

```
private async fetchNewWithdrawals(lastProcessedBlockHeight: number):
    ↪  Promise<EnvioWithdrawalEvent[]> {
  try {
      logger.debug(PROCESS, `Querying withdrawals after block: ${lastProcessedBlockHeight}`);

      const response: EnvioGraphQLResponse = await this.client.request(this.WithdrawalsQuery, {
        lastBlockHeight: lastProcessedBlockHeight,
        limit: this.batchSize,
        env: this.environment.replace('-testnet', '')
      });

      const events = response.FastBridgeWithdrawalEvent || [];

      return events.map((event) => ({
        ...event,
        source: 'envio' as const
      }));
  } catch (error) {
    logger.error(PROCESS, 'Failed to fetch withdrawals from Envio:', error);
    throw error;
  }
}
```

### Remediation

Update the logic to consider the edge case when the number of withdrawal events exceeds the configured `batchSize`.

### Patch

Resolved in PR#272.

## Floating-Point Precision Calculation Risk HIGH OS-FFB-ADV-03

### Description

In the current implementation, there are multiple instances where the codebase utilizes TypeScript's `number` type for arithmetic. Since this relies on floating-point representation, it may introduce rounding errors. These imprecisions may have significant downstream effects, such as incorrect token amount conversions and faulty comparison results between values.

### Remediation

Ensure calculations utilize `BigInt`.

### Patch

Resolved in PR#289 and PR#437.

## Failure to Decrement Relayer Count HIGH OS-FFB-ADV-04

### Description

`main::remove_relayer` in `fast-bridge` deletes a relayer's address but does not decrement `relayer_count`, resulting in an inconsistent state. This mismatch may misleadaffect quorum or threshold calculation since they rely on the total count of active relayers. As a result, the system may expect more signatures than are actually available, breaking approval logic.

*>_ contracts/sway/fast-bridge/src/main.sw* RUST

```rust
#[storage(read, write)]
fn remove_relayer(relayer: Address) {
    only_owner();
    storage.relayer_index.remove(relayer);
}
```

### Remediation

Decrement `relayer_count` in `remove_relayer`.

### Patch

Resolved in PR#211.

## Inconsistent Withdrawal Hash Encoding MEDIUM OS-FFB-ADV-05

### Description

`generateRequestHash` in `withdrawals.task` utilizes `AbiCoder.encode`, which applies padded ABI encoding, which results in inconsistencies with the on-chain events, where the withdrawal hash is calculated utilizing `encodePacked`. This results in mismatched hash outputs between off-chain and on-chain computations. As a result, withdrawal requests may not reflect their on-chain counterparts.

>_ *src/tasks/withdrawals.task.ts* TYPESCRIPT

```typescript
function generateRequestHash(event: SentioWithdrawalEvent): string {
  // Should always match the asset-registry contract: keccak256((destination_chain, asset_id,
      ↪  recipient, amount, nonce))
  return ethers.keccak256(
    ethers.AbiCoder.defaultAbiCoder().encode(
      ['uint32', 'bytes32', 'bytes32', 'uint64', 'uint64'],
      [event.destination_chain, event['asset_id.bits'], event.recipient, BigInt(event.amount),
          ↪  BigInt(event.nonce)]
    )
  );
}
```

### Remediation

Utilize `encodePacked` for withdrawal hash encoding to ensure consistency with on-chain events.

### Patch

Resolved in PR#289.

## Incomplete Withdrawal Event Verification MEDIUM OS-FFB-ADV-06

### Description

`verifyWithdrawalEvent` in `withdrawalVerification` verifies that a specific Fuel transaction ( `txId` ) contains a FastBridge withdrawal event that matches the expected withdrawal data provided as `expectedEvent` . However, it only verifies the first `FastBridgeWithdrawalEvent` in a transaction utilizing `.find` , ignoring any additional withdrawal events. This implies that if multiple events exist, unverified ones will go undetected. As a result, the function may incorrectly mark a transaction as valid even when other withdrawal events are inconsistent, falsely returning `verified: true` .

>_ *src/utils/withdrawalVerification.ts* TYPESCRIPT

```typescript
export async function verifyWithdrawalEvent(
  txId: string,
  expectedEvent: UnifiedWithdrawalEvent,
  fuelProvider: Provider
): Promise<VerificationResult> {
  try {
    [...]
    // Find all LOG_DATA receipts and filter
    const allLogDataReceipts =
      transaction.status.receipts?.filter((receipt: any) => receipt.receiptType === 'LOG_DATA')
          ↪  || [];

    const logDataReceipt = allLogDataReceipts.find((receipt: any) => receipt.rb ===
        ↪  WITHDRAWAL_LOG_TYPE_ID);
    [...]
    return { verified };
  } catch (error) {[...]}
}
```

### Remediation

Ensure the function verifies all matching events to ensure complete validation.

### Patch

Resolved in PR#336.

## Race Condition on UTXO Spending due to Shared Signer MEDIUM OS-FFB-ADV-07

### Description

Currently, the `FastBridge` contract is instantiated utilizing `context.fuelSigner`, which is responsible for submitting deposit-related transactions to the Fuel chain. However, the `GasPublisherTask` also utilizes the same signer ( `context.fuelSigner` ) to periodically publish gas price updates. This may result in issues where the two clients run into race conditions when fetching UTXOs (Unspent Transaction Outputs) to spend as gas. Both processes may attempt to consume overlapping UTXOs at the same time. If both clients fetch and attempt to spend the same UTXO, one transaction will fail due to a double-spend conflict.

### Remediation

Utilize separate signers for deposit processing and gas publishing to avoid conflicts.

### Patch

Resolved in PR#286.

## Possibility of Stale oracle configuration MEDIUM OS-FFB-ADV-08

### Description

The offchain gas publisher loads the oracle configuration only once and never refreshes it. If the on-chain configuration is upgraded, the offchain task may continue utilizing outdated thresholds or intervals. This may result in delayed gas updates or overly frequent updates, resulting in transaction failures or inefficiency.

### Remediation

Ensure the offchain component is always synchronized with on-chain configuration changes.

### Patch

Resolved in PR#352.

## Hardcoded Fuel Chain ID LOW OS-FFB-ADV-09

### Description

The `fuel_chain_id` is currently not configurable as it is hardcoded in `processDeposit`. Moreover, it is hardcoded to an incorrect value of zero which is the ID for the Fuel testnet. This will result in deposits from other networks to be incorrectly labeled as testnet transactions.

>_ *src/utils/withdrawalVerification.ts* TYPESCRIPT

```typescript
export function processDeposit(baseMessenger: Messenger, txService: FuelTransactionService,
    ↪  chainConfig: ChainConfig) {
  return async (block: number) => {
    try {
      [...]
      const queuedDeposits = await Promise.all(
        deposits.map(async ({ args }) => {
          [...]
          const depositApprovalInput: DepositApprovalInput = {
            [...]
            fuel_chain_id: 0,
            blockNumber: block
          };
          [...]
        })
      );

      return queuedDeposits.filter((deposit) => deposit !== null);
    } catch (error) {[...]}
  };
}
```

### Remediation

Update the code such that `fuel_chain_id` is configurable, ensuring it accurately reflects the target Fuel network in utilization.

### Patch

Resolved in PR#252.

## Incorrect Decimal Precision Handling LOW OS-FFB-ADV-10

### Description

`decimals` in `SRC20` implementation currently retrieves and returns the token's native decimal value from the remote chain (via `details.decimals` in the external registry), instead of enforcing the fixed decimal precision of bridged Fuel assets (which should always be 9). This may result in inconsistent decimal precision for bridged tokens.

>_ *contracts/sway/asset-registry/src/main.sw* SWAY

```
#[storage(read)]
fn decimals(asset: AssetId) -> Option<u8> {
    [...]
    let registry = abi(AssetRegistry, registry_id);
    match registry.get_token_details(sub_id) {
        Some(details) => Option::Some(details.decimals),
        None => Option::None,
    }
}
```

### Remediation

Ensure the function returns a constant value of 9 for consistency with Fuel's fixed 9-decimal format.

### Patch

Resolved in PR#228.

# 05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID | Description |
| --- | --- |
| OS-FFB-SUG-00 | Sentio may intermittently miss events in subcalls when the target contract is not the first in a transaction, raising reliability concerns for production systems. |
| OS-FFB-SUG-01 | The transaction limit check may be bypassed by splitting transfers into multiple calls within a single transaction, rendering the restriction ineffective without aggregate tracking. |
| OS-FFB-SUG-02 | Automatically lowering the threshold on signer removal may weaken multi-sig security and result in inconsistent behavior during signer replacements. |
| OS-FFB-SUG-03 | Suggestions regarding ensuring adherence to coding best practices. |
| OS-FFB-SUG-04 | Recommendation for refactoring the codebase for improved functionality and efficiency. |
| OS-FFB-SUG-05 | Several discrepancies and ambiguities in the existing codebase should be properly addressed for consistency and clarity. |

## Indexer Event Reliability Concerns OS-FFB-SUG-00

### Description

The issue concerns the reliability of the Sentio indexer when tracking events from smart contracts. Empirical tests show that Sentio may intermittently miss events emitted by a contract if it is not the first contract in the transaction inputs, creating non-deterministic gaps in event data. Since Sentio is partially closed-source, the exact cause of these missed events is opaque, rendering it difficult to guarantee correctness. This raises potential risks for production systems that rely on these events for state updates or triggers.

### Remediation

Ensure to contact the indexer developers for clarification and implement thorough integration and fuzz testing to detect and handle such inconsistencies before deployment.

### Patch

This issue was acknowledged by Fuel.

## Ineffective Transaction Limit Enforcement OS-FFB-SUG-01

### Description

`main::_check_transaction_limit` in `asset_registry` is not fully functional, given that a user may invoke the `asset_registry` contract several times in a single transaction through a script of contract. As it only enforces a per-call limit, users may bypass it by splitting transfers into smaller chunks within the same transaction. Since it does not track aggregate amounts, the restriction becomes ineffective in practice. This undermines its purpose, as users may still exceed the intended cap.

>_ *contracts/sway/asset-registry/src/main.sw* RUST

```rust
#[storage(read)]
fn _check_transaction_limit(asset_id: AssetId, amount: u64) {
    let tx_limit = storage.asset_tx_limits.get(asset_id).try_read().unwrap_or(0);
    if tx_limit > 0 {
        require(amount <= tx_limit, "Transaction amount exceeds limit");
    }
}
```

### Remediation

Reconsider the function logic and its intent.

### Patch

This issue was acknowledged by Fuel.

## Automatic Threshold Adjustment Risk OS-FFB-SUG-02

### Description

`Outpost::removeSigner` automatically lowers the threshold if it exceeds the new signer count, which may unintentionally weaken multi-sig security by reducing the number of approvals required. This creates inconsistent outcomes depending on whether the admin removes then adds, or adds then removes, when replacing a signer. Such automatic adjustments may complicate contract management and increase the risk of misconfiguration.

>_ *contracts/Outpost.sol* SOLIDITY

```solidity
function removeSigner(address signer) external onlyRole(DEFAULT_ADMIN_ROLE) {
    [...]
    if (threshold > signerCount && signerCount > 0) {
        threshold = signerCount;
        emit ThresholdUpdated(threshold + 1, threshold);
    }
    emit SignerRemoved(signer, signerCount);
}
```

### Remediation

Avoid auto-adjustment, and instead, require the admin to explicitly update the threshold after signer changes.

### Patch

This issue was acknowledged by Fuel.

## Code Maturity OS-FFB-SUG-03

### Description

1. `Messenger::_deposit` utilizes `transfer` to send `ETH`, which only forwards 2,300 gas. This may break deposits if the vault is a contract requiring more gas or if future EVM changes increase gas costs. Replacing `transfer` with `call` ensures compatibility and prevents unexpected reverts.

   *>_ contracts/Messenger.sol* SOLIDITY

   ```
   function _deposit(bytes32 to, address token, uint256 value, bool isContract) internal {
       [...]
       if (token == NATIVE_ETH) {
           require(msg.value == value, InsufficientAmountError());
           payable(vault).transfer(value);
       }
       [...]
   }
   ```

2. The codebase contains several instances of unutilized functions/declarations and TODOs. Remove unutilized and dead code, and ensure all TODOs are addressed appropriately.
3. The events in `wrapped_asset_minter` fail to adhere to the `src20` standard and it defines a custom `Metadata` enumeration instead of the `src7` standard.
4. The fuel-toolchain currently pins an older compiler version (`forc 0.66.5`), which predates several important optimization fixes introduced in newer releases. Update to the latest stable release for compatibility and to benefit from optimization fixes.
5. The Fuel fast-bridge utilizes a redundant `ROLE_ADMIN` constant which duplicates the functionality of OpenZeppelin's `DEFAULT_ADMIN_ROLE`. Directly utilize `DEFAULT_ADMIN_ROLE` to maintain clarity and align with standard `AccessControl` practices.

### Remediation

Implement the above-mentioned suggestions.

### Patch

1. Issue #1 resolved in 0625d93.
2. Issue #2 resolved in PR#437 and PR#392.
3. Issue #3 was partially resolved in PR#359 and partially acknowledged by Fuel.

4. Issue #4 was acknowledged by Fuel.
5. Issue #5 was acknowledged by Fuel.

## Code Refactoring OS-FFB-SUG-04

### Description

1. The current implementation of `handleFailedWithdrawal` in `EVMTransactionService` permanently removes a withdrawal after three consecutive failures, effectively skipping it. This may result in unprocessed bridge withdrawals and user funds remaining locked on the source chain. Since failed withdrawals are neither persisted nor retried, recovery requires manual intervention or reprocessing of the bridging event. A retry mechanism should be implemented instead of removing the withdrawal from the queue.

   *>_ src/infra/services/EVMTransactionService.ts* TYPESCRIPT

   ```
   private async handleFailedWithdrawal(withdrawal: QueuedWithdrawal): Promise<void> {
     withdrawal.retryCount++;
     if (withdrawal.retryCount >= MAX_RETRIES) {
       logger.error(
         EVMTransactionService.SERVICE_NAME,
         `Withdrawal ${withdrawal.request_hash} exceeded max retries (${MAX_RETRIES}),
             ↪  removing from queue`
       );
       this.queue.delete(withdrawal.request_hash);
       return;
     }
     [...]
   }
   ```

2. In `DatabaseService::getProcessedWithdrawals`, the function annotation incorrectly lists `fuelstreams` as a possible event source, but the actual event indexing systems utilized are `envio` and `sentio`. Update the annotation to `'envio' | 'sentio'`.

   *>_ src/infra/services/DatabaseService.ts* TYPESCRIPT

   ```
   public getProcessedWithdrawals(limit: number = 50, source?: 'fuelstreams' | 'sentio'):
       ↪  ProcessedWithdrawal[] {
     [...]
   }
   ```

3. The existing implementation counts the approvals from removed signers toward the final threshold, allowing removed and potentially compromised keys to approve transactions. Only active signers should be counted to ensure valid and up-to-date authorization during execution.

4. The initialization function in the Sway fast-bridge contract does not set a relayer approval threshold, leaving it at zero by default until re-configured later. Set a threshold during the initialization process within `initialize`.

>_ *contracts/sway/fast-bridge/src/main.sw* SWAY

```
impl FastBridge for Contract {
    /// Initializes the fast bridge contract with the configured owner, registry, and
        ↪  chain ID.
    #[storage(read, write)]
    fn initialize() {
        initialize_ownership(INITIAL_OWNER);
        storage.registry_contract.write(INITIAL_REGISTRY);
        storage.fuel_chain_id.write(INITIAL_FUEL_CHAIN_ID);
    }
    [...]
}
```

5. It is not possible for admins to replenish the bridge's daily withdrawal cap once it is exhausted. This allows an attacker to repeatedly bridge small amounts back and forth to quickly consume the limit, blocking all further withdrawals, resulting in a denial-of-service scenario. Consider adding functionality to enable the admin to replenish the withdrawal cap.
6. The absence of a rebalancing mechanism may create uneven liquidity distribution, where some assets become depleted while others remain idle. This results in failed unwrap operations and inefficient asset utilization across the bridge.

### Remediation

Incorporate the refactors in the codebase.

### Patch

1. Issue #1 resolved in PR#310.
2. Issue #2 resolved in PR#347.
3. Issue #3 was acknowledged by Fuel.
4. Issue #4 was partially resolved in PR#182 and the remaining instance was acknowledged by Fuel.
5. Issue #5 was acknowledged by Fuel.
6. Issue #6 was acknowledged by Fuel.

## Code Inconsistencies OS-FFB-SUG-05

### Description

1. The gas rounding logic differs between OP Stack and Ethereum. `OPStackGasPriceOracle::calculateTotalGasFee` rounds up to the nearest Gwei, while in Ethereum, `EVMGasMonitor::getCurrentGasPriceGwei` truncates fractional values. This inconsistency may result in a slight underestimation of gas fees on Ethereum compared to OP Stack. Ensure consistency during gas rounding.
2. The withdrawal hash computation differs between `asset_registry` and `outpost`, resulting in cross-chain inconsistencies. Fuel includes `destination_chain` and `asset_id` in `withdraw_via_fast_bridge`, while Ethereum utilizes `block.chainid` and `token` in `approveWithdrawal`. Ensure consistency during hash computation.
3. Currently, `get_gas_oracle` and `get_relayer_treasury` return zero values when unset, which creates ambiguity between an uninitialized state and a deliberately set value. This may mask configuration errors and result in unsafe assumptions in downstream logic. Returning an `Option` will force explicit handling of unset values, improving clarity and safety. Similarly, `get_withdrawal_fee` should revert instead of returning zero when the lookup fails.
4. The current wrapping and unwrapping logic assumes equal decimal precision between canonical and wrapped assets, which may result in mismatched token amounts. If a canonical asset utilizes fewer or more decimals, users may receive an incorrect quantity of wrapped tokens or face balance discrepancies. This breaks 1:1 parity and may give rise to accounting or redemption inconsistencies. Add decimal conversion logic for `wrap_canonical_asset` and `unwrap_to_canonical_asset` to prevent issues when the canonical asset uses a different decimal from bridged assets.

### Remediation

Address the issues stated above.

### Patch

1. Issue #1 resolved in PR#352.
2. The Hash calculation on the relayer was resolved in PR#437.
3. Issue #3 resolved in PR#358.
4. Issue #4 resolved in PR#224.

# 06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

**CRITICAL**

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

**HIGH**

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

**MEDIUM**

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

**LOW**

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

**INFO**

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

# 07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.